# ASM 591 In-Lab Assignment: Soil Temperature Dashboards (Grafana + PostgreSQL)

## Overview

In this lab, you will load `soil_temperature.csv` into a PostgreSQL database, connect the database to Grafana, and build a dashboard with four visualizations. You will:

1. Inspect a CSV file and infer column meanings and data types.
2. Write a SQL `CREATE TABLE` statement (you may use generative AI tools such as ChatGPT).
3. Use `csv_to_sql.py` to generate a bulk `INSERT` script and populate the table.
4. Add PostgreSQL as a Grafana data source and create a dashboard with required panels.

## Files and tools

- **Dataset:** `soil_temperature.csv`
- **Script:** `csv_to_sql.py` (generic CSV → bulk SQL generator)
- **Database:** PostgreSQL
- **Dashboard:** Grafana

## Learning objectives

By the end of this lab, you should be able to:

- choose appropriate SQL types for timestamped sensor data,
- bulk-load CSV data into a relational database,
- write SQL queries suitable for Grafana panels (time-series and bar charts),
- use Grafana macros (e.g., `$__timeFilter`) to make dashboards interactive.

## Getting connected

### Connecting to the PostgreSQL database (Beekeeper Studio)

Use Beekeeper Studio to connect to the class PostgreSQL database.

**Connection details:**

- **Host:** `aglab`
- **Port:** `5432`
- **Database:** `aglab`
- **Username:** `studentX` (`X` will be provided by the instructor)
- **Password:** (provided by the instructor)

**Steps:**

1. Open Beekeeper Studio and click **New Connection**.
2. Choose **PostgreSQL**.
3. Enter the connection details above.
4. Click **Test Connection**, then click **Connect**.

### Connecting to Grafana at `aglab:3000` (Tailscale)

Grafana is available at `http://aglab:3000`. You must join the course Tailscale network to access it.

**Step 1: Join the course Tailscale network**

1. Open the Tailscale invite link or use the invite sent to your email to access the machine:
   - `https://login.tailscale.com/admin/invite/PpBVZMpGqJENyeEyDfZY11`
2. Confirm that Tailscale shows you are **connected**.

**Step 2: Open Grafana**

- In a browser, go to: `http://aglab:3000`
- Log in with:
  - **Username:** `studentX` (same as database username)
  - **Password:** (same as your database password)

## Part A: Inspect the CSV and infer schema (before writing SQL)

Open `soil_temperature.csv` and answer briefly:

1. What are the column names?
2. What does each column represent?
3. Which columns are identifiers? Which are measurements?
4. Which columns should be numeric vs. text?
5. Does the timestamp include a timezone offset (e.g., `+00`)?

**Expected columns (from the file):**

```
ts, device_id, depth_cm, temp_c
```

## Part B: Create the SQL table (you may use ChatGPT)

Using your analysis, write a `CREATE TABLE` statement. Your table must:

- store timestamps correctly (timezone-aware),
- store the device identifier,
- store depth in centimeters,
- store temperature in Celsius with decimals,
- enforce a sensible primary key.

### Recommended schema (reference)

You may use this as a target if you are unsure:

```
CREATE TABLE soil_temperature (
  ts TIMESTAMPTZ NOT NULL,
  device_id TEXT NOT NULL,
  depth_cm INTEGER NOT NULL,
  temp_c NUMERIC(6,2),

  PRIMARY KEY (ts, device_id, depth_cm)
);

CREATE INDEX soil_temperature_ts_idx ON soil_temperature (ts);
CREATE INDEX soil_temperature_device_ts_idx ON soil_temperature (device_id, ts);

SELECT create_hypertable('soil_temperature', 'ts');
```

## Part C: Convert CSV to a bulk INSERT and load the data

### 1) Generate a bulk SQL file

Use `csv_to_sql.py` to generate a **single bulk INSERT** file.
**Command (run in a terminal):**

```
python csv_to_sql.py soil_temperature.csv \
  --table soil_temperature \
  --cols ts,device_id,depth_cm,temp_c \
  --text-cols ts,device_id \
  --int-cols depth_cm \
  --numeric-cols temp_c \
  > soil_temperature_bulk.sql
```

### 2) Load the data into PostgreSQL

- Connect to your database using Beekeeper Studio.
- Open the SQL query editor and paste the contents of `soil_temperature_bulk.sql` and execute.

### 3) Verify the data

Run a quick check or view the table from the menu on the left:

```
SELECT COUNT(*) FROM soil_temperature;
```

## Part D: Connect Grafana to PostgreSQL

1. In Grafana: **Connections** → **Data sources** → **Add data source**.
2. Select **PostgreSQL**.
3. Fill in the connection settings:
   - **Host:** `aglab-timescaledb:5432`

- **Database:** `aglab`
- **User:** `studentX` (`X` provided by the instructor; same as your Grafana username)
- **Password:** (provided by the instructor; same as your Grafana password)
- **TLS/SSL Mode: Disable**

4. Click **Save & Test**. You should see a success message.

## Dashboard requirements (4 tasks)

Create a new dashboard named **Soil Temperature Lab**. Add one panel per task below. Each panel must have:

- a clear title,
- labeled axes (when applicable)

### Task 1: Line graph (All sensors)

**Goal:** Plot temperature over time for all sensors.
**Panel type:** Time series (line)
**Important:** The `WHERE $__timeFilter(ts)` condition is required so the query respects Grafana's dashboard time-range selector (top-right). Without it, Grafana may attempt to plot the entire table and the panel will not properly follow the selected timeframe.
**Also:** Use `ORDER BY ts` so points are returned in chronological order.
**Write a SQL query that returns:**

- a timestamp column for Grafana (`ts`)
- a numeric temperature column (`temp_c`)
- a `WHERE $__timeFilter(ts)` clause (required)
- results sorted by time (`ORDER BY ts`)

**Template (fill in the blanks):**

```
SELECT
  _________ AS time,
  _________ AS value
FROM soil_temperature
WHERE $__timeFilter(_________)
ORDER BY _________;
```

**Note (important for this lab):** The dataset is not live. If your dashboard time range is set to "Last 6 hours" or "Last 24 hours", the panels may initially appear empty. Set the dashboard time range (top-right) to a window that includes the CSV dates. For this lab, use a custom range (UTC Time):

- **From:** `2025-07-01 08:00:01`
- **To:** `2025-08-01 07:59:18`

Then click **Apply**.

### Task 2: Line graph (One chosen sensor)

**Goal:** Plot temperature over time for **only one sensor** (you choose which `device_id`).
**Panel type:** Time series (line)
**Important:** The `WHERE $__timeFilter(ts)` condition is required so the query respects Grafana's dashboard time-range selector (top-right). Without it, Grafana may attempt to plot the entire table and the panel will not properly follow the selected timeframe.

**Also:** Use `ORDER BY ts` so points are returned in chronological order.
**Write a SQL query that returns:**

- a timestamp column for Grafana (`ts`)
- a numeric temperature column (`temp_c`)
- a `WHERE $__timeFilter(ts)` clause (required)
- a filter that restricts results to **one** `device_id`
- results sorted by time (`ORDER BY ts`)

**Template (fill in the blanks):**

```
SELECT
  _________ AS time,
  _________ AS value
FROM soil_temperature
WHERE $__timeFilter(_________)
  AND device_id = '________________________'
ORDER BY _________;
```

**Note (important for this lab):** The dataset is not live. If your dashboard time range is set to "Last 6 hours" or "Last 24 hours", the panels may initially appear empty. Set the dashboard time range (top-right) to a window that includes the CSV dates. For this lab, use a custom range (UTC Time):

- **From:** `2025-07-01 08:00:01`
- **To:** `2025-08-01 07:59:18`

Then click **Apply**.

### Task 3: Bar chart (1-day buckets, all sensors)

**Goal:** Create a bar chart using **1-day time buckets** that summarizes temperature for **all sensors combined**. You may choose an aggregation such as `AVG(temp_c)` or `MAX(temp_c)`.
**Panel type:** Bar chart
**Write a SQL query that returns:**

- a 1-day "bucketed" timestamp column as `time`
- an aggregated temperature column as `value` (e.g., `AVG(temp_c)`)
- a `WHERE $__timeFilter(ts)` clause (required)
- results sorted by the bucketed time

**Template (fill in the blanks):**

```
SELECT
  time_bucket('1 day', _________) AS time,
  _________(temp_c) AS value
FROM soil_temperature
WHERE $__timeFilter(_________)
GROUP BY 1
ORDER BY 1;
```

**Notes:**

- In the panel options, select the Bar Chart visualization.
- This produces **one bar per day** representing the chosen aggregation across **all sensors**.
- If you want to compare sensors separately, you would include `device_id` as a series label and add it to the `GROUP BY` clause (not required for this task).

**Note (important for this lab):** The dataset is not live. If your dashboard time range is set to "Last 6 hours" or "Last 24 hours", the panels may initially appear empty. Set the dashboard time range (top-right) to a window that includes the CSV dates. For this lab, use a custom range (UTC Time):

- **From:** `2025-07-01 08:00:01`
- **To:** `2025-08-01 07:59:18`

Then click **Apply**.

### Task 4: Histogram with a dashboard variable (choose the sensor)

**Goal:** Create a histogram of temperature values where the viewer can choose a sensor from a dropdown and the histogram updates.
**Panel type:** Histogram
**Step 1 (Create a dashboard variable):** Create a variable named `sensor` that lists available device IDs.

- Go to: **Dashboard** → **Settings** → **Variables** → **Add variable**
- Name: `sensor`
- Type: `Query`
- Data source: PostgreSQL
- Query:

```
SELECT DISTINCT device_id
FROM soil_temperature
ORDER BY device_id;
```

**Step 2 (Write a SQL query that returns):**

- a numeric temperature column (`temp_c`) to build the histogram from
- a `WHERE $__timeFilter(ts)` clause (required so the histogram reflects the selected timeframe)
- a filter that uses the dashboard variable `$sensor`

**Template (fill in the blanks):**

```
SELECT
  _________ AS value
FROM soil_temperature
WHERE $__timeFilter(_________)
  AND device_id = '$sensor';
```

**Step 3 (Configure the histogram bins in Grafana):**

- In the panel options, enable histogram mode (or select the Histogram visualization).
- Ensure the x-axis represents temperature values and the y-axis represents counts.

**Note (important for this lab):** The dataset is not live. If your dashboard time range is set to "Last 6 hours" or "Last 24 hours", the panels may initially appear empty. Set the dashboard time range (top-right) to a window that includes the CSV dates. For this lab, use a custom range (UTC Time):

- **From:** `2025-07-01 08:00:01`
- **To:** `2025-08-01 07:59:18`

Then click **Apply**.
**Check:** Change the dashboard time range and the `sensor` dropdown. Confirm that the histogram shape changes based on (i) timeframe and (ii) chosen sensor.

## What to submit

Submit the following (as instructed by your course platform):

1. **Part A (CSV analysis):** Your brief answers about the CSV columns (names, meanings, identifier vs. measurement, data types, and whether `ts` includes a timezone offset).
2. **Part B (Schema):** Your final `CREATE TABLE` SQL (copy/paste).
3. **Part C (Loading):** The exact `csv_to_sql.py` command you ran to generate the bulk insert file, and the verification query result (e.g., `SELECT COUNT(*) FROM soil_temperature;`).
4. **Dashboard queries:** The SQL query you used for each panel (Task 1–Task 4).
5. **Screenshots:** One screenshot per panel (Task 1–Task 4) showing the panel title and the visualization. For Task 4, include a screenshot showing the `sensor` variable dropdown (or variable settings) and the histogram.

## Grading checklist

| Item | Criteria | Points |
|---|---|---|
| Part A (CSV analysis) | Brief, correct interpretation of columns and data types | 10 |
| Schema | Correct types; sensible primary key; table created successfully | 15 |
| Load | Bulk insert generated and data loaded; verification query shown | 15 |
| Task 1 | Line chart for all sensors (correct axes and formatting) | 15 |
| Task 2 | Line chart for one chosen sensor (correct filtering) | 15 |
| Task 3 | Bar chart with 1-day buckets for all sensors combined (aggregation OK) | 15 |
| Task 4 | Histogram for a selected sensor using a dashboard variable | 15 |